STEP 1 — Prerequisites

1. Check Python

Open VS Code → Terminal → New Terminal

Run:

python --version

You need Python 3.10 or newer.

If it says *not recognized*, install Python:

[Download Python]

Windows installation: Make sure to tick “Add Python to PATH” during installation.

2. Check VS Code

If VS Code is already open, you're ready.

Otherwise, download:

[Download VS Code]

3. Check Git

In VS Code → Terminal:

git --version

If a version appears → ✅ Ready.

If it says *not recognized*, install:

[Download Git]

4. Get Gemini API Key

Open:

Google AI Studio

Navigate to:

Get API key → Create API key

Keep the key safe. Do not share it.

STEP 2 — Create Project & Virtual Environment

1. Open Terminal

2. Create the project

mkdir ai-resume-assistant
cd ai-resume-assistant

3. Create the backend folder

mkdir backend
cd backend

4. Create the virtual environment inside backend

run in terminal : python -m venv venv

Now the structure is:

ai-resume-assistant/
└── backend/
  └── venv/

5. Activate the virtual environment

Windows:run in terminal

venv\Scripts\activate

You should see:

(venv)

at the beginning of the terminal.

6. Upgrade pip

python -m pip install --upgrade pip

Final structure

ai-resume-assistant/
└── backend/
  └── venv/

STEP 3 — Install Libraries & Create Files

1. Install required libraries

Run in terminal :

pip install fastapi uvicorn pypdf2 google-genai chromadb streamlit python-dotenv python-multipart

2. Create the project files

In VS Code, look at the **Explorer** on the left.

Right-click `backend` → **New File**.

Create these files:

Your structure should be:

```
ai-resume-assistant/
└── backend/
    ├── venv/
    ├── main.py
    ├── streamlit_app.py
    ├── rag_service.py
    ├── chunking.py
    ├── embedding_service.py
    ├── vector_store.py
    ├── requirements.txt
    └── .env
```

3. Create `.env`

Open:

.env

Add:

GEMINI_API_KEY=your_api_key_here

Replace `your_api_key_here` with your actual Gemini API key.

Do not share this key.

4. Save installed libraries

Run:

pip freeze > requirements.txt

This saves the installed packages into `requirements.txt`.

STEP 4 — Build & Test FastAPI Backend

1. Open `main.py`

In VS Code:

Explorer → backend → main.py

Add:

```
from fastapi import FastAPI

app = FastAPI()



@app.get("/")
def home():
  return {
     "message": "AI Resume Assistant API is running"
  }
```

2. Save the file

Press:

Ctrl + S

3. Start the FastAPI server

Make sure the terminal is inside:

ai-resume-assistant/backend

and `(venv)` is visible.

run in terminal

uvicorn main:app --reload

You should see:

Uvicorn running on http://127.0.0.1:8000

4. Test the API

Open your browser and go to:

http://127.0.0.1:8000

You should see:

```
{
 "message": "AI Resume Assistant API is running"
}
```

5. Open FastAPI documentation

Go to: **http://127.0.0.1:8000/docs**

You should see the **FastAPI Swagger UI**.

At this stage, you should see:

GET /

The project guide uses the same `/` endpoint and Swagger documentation.

Important

Keep this terminal running. Don't close it.

STEP 5 — Upload PDF & Extract Text

1. Open `main.py`

Replace the existing code with:

```
from fastapi import FastAPI, UploadFile, File
from PyPDF2 import PdfReader

app = FastAPI()

@app.get("/")
def home():
  return {
    "message": "AI Resume Assistant API is running"
  }

@app.post("/upload")
def upload_resume(file: UploadFile = File(...)):
  reader = PdfReader(file.file)

  text = ""

  for page in reader.pages:
    page_text = page.extract_text()
```

```
        if page_text:
            text += page_text

    return {
        "filename": file.filename,
        "pages": len(reader.pages),
        "text": text
    }
```

2. Start FastAPI again

Run: **uvicorn main:app --reload**

3. Upload your resume

In Swagger:

POST /upload → Try it out → Choose File

Select your resume PDF.

Then click:

Execute

4. Check the response

You should get something similar to:

```
{
 "filename": "resume.pdf",
 "pages": 2,
 "text": "Your extracted resume text..."
}
```

STEP 6 — Create Chunking Function & Connect to `main.py`

1. Open `chunking.py`

Go to: **VS Code** → **Explorer** → **backend** → `chunking.py`

Add:

```
def chunk_text(text, chunk_size=500):
    chunks = []

    for i in range(0, len(text), chunk_size):
        chunk = text[i:i + chunk_size]
        chunks.append(chunk)

    return chunks
```

2. Open `main.py`

At the top, add:

```
from chunking import chunk_text
```

Your imports should now be:

```
from fastapi import FastAPI, UploadFile, File
from PyPDF2 import PdfReader
from chunking import chunk_text
```

3. Update the `/upload` function

Replace your current `/upload` function with:

```
@app.post("/upload")
def upload_resume(file: UploadFile = File(...)):
  reader = PdfReader(file.file)

  text = ""

  for page in reader.pages:
    page_text = page.extract_text()

    if page_text:
      text += page_text

  # Split resume text into chunks
  chunks = chunk_text(text)

  return {
    "filename": file.filename,
    "pages": len(reader.pages),
    "text": text,
    "chunks": chunks,
    "chunk_count": len(chunks)
  }
```

STEP 7 — Create Gemini Embedding Service

1. Open `embedding_service.py`

Go to:

VS Code → Explorer → backend → `embedding_service.py`

Add:

```
import os

from google import genai
from dotenv import load_dotenv

load_dotenv()

client = genai.Client(
  api_key=os.getenv("GEMINI_API_KEY")
)

def generate_embedding(text):
  response = client.models.embed_content(
    model="gemini-embedding-001",
    contents=text
  )

  return response.embeddings[0].values
```

2. Connect it to `main.py`

Open:

`backend/main.py`

Add this import:

from embedding_service import generate_embedding

3. Generate embeddings for every chunk

Inside your `/upload` function, **after**:

chunks = chunk_text(text)

add:

embeddings = []

for chunk in chunks:
  vector = generate_embedding(chunk)
  embeddings.append(vector)

4. Update the return

Change your return to:

return {
  "filename": file.filename,
  "pages": len(reader.pages),
  "text": text,
  "chunks": chunks,
  "chunk_count": len(chunks),
  "embedding_count": len(embeddings)
}

STEP 8 — Create ChromaDB & Store Embeddings

1. Open `vector_store.py`

Go to:

VS Code → Explorer → backend → `vector_store.py`

Add:

```
import chromadb

client = chromadb.PersistentClient(
  path="./chroma_db"
)

collection = client.get_or_create_collection(
  name="resume_collection"
)

def store_embeddings(chunks, embeddings):
  ids = []

  for i in range(len(chunks)):
    ids.append(f"chunk_{i}")

  collection.add(
    ids=ids,
    documents=chunks,
    embeddings=embeddings
  )
```

This creates:

chroma_db/
└── resume_collection

and stores each chunk with its embedding.

2. Connect it to `main.py`

Open:

`backend/main.py`

Add in Top::

from vector_store import store_embeddings

3. Store the embeddings

Inside `/upload`, after:

embeddings = []

for chunk in chunks:
  vector = generate_embedding(chunk)
  embeddings.append(vector)

add:

store_embeddings(chunks, embeddings)

STEP 9 — Create RAG Search

1. Open `vector_store.py`

Add this **below** `store_embeddings()`:

```
def search_embeddings(query_embedding, top_k=3):
  results = collection.query(
    query_embeddings=[query_embedding],
    n_results=top_k
  )

  return results
```

Here:

top_k=3

means:

> Return the **3 most relevant chunks**.

2. Update `main.py`

Change:

from vector_store import store_embeddings

to:

from vector_store import store_embeddings, search_embeddings

3. Add the Search API

In `main.py`, below the `/upload` function, add:

```
@app.post("/search")
def search_resume(question: str):

  # Convert question into an embedding
  query_embedding = generate_embedding(question)

  # Search ChromaDB
  results = search_embeddings(query_embedding)

  return {
     "question": question,
     "results": results["documents"]
  }
```

STEP 10 — Generate the AI Answer

1. Open `rag_service.py`

Go to:

VS Code → Explorer → backend → `rag_service.py`

Add:

```
import os
from google import genai
from dotenv import load_dotenv

load_dotenv()
```

```
client = genai.Client(
  api_key=os.getenv("GEMINI_API_KEY")
)

def generate_answer(question, context):

  prompt = f"""
You are an AI Resume Assistant.

Answer the user's question using only the information
provided in the resume context.

Resume Context:
{context}

Question:
{question}

If the answer is not available in the resume context,
say that the information is not available in the resume.
"""

  response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
  )

  return response.text
```

2. Connect it to `main.py`

Add this import:

from rag_service import generate_answer

3. Update `/search`

Replace your current `/search` function with:

```
@app.post("/search")
def search_resume(question: str):

    # Convert question into an embedding
    query_embedding = generate_embedding(question)

    # Search ChromaDB
    results = search_embeddings(query_embedding)

    # Get relevant chunks
    relevant_chunks = results["documents"][0]

    # Combine chunks into one context
    context = "\n\n".join(relevant_chunks)

    # Generate AI answer
    answer = generate_answer(
        question,
        context
    )
```

```
    return {
        "question": question,
        "context": relevant_chunks,
        "answer": answer
    }
```

STEP 11 — Build the Streamlit UI

1. Open `streamlit_app.py`

Go to:

VS Code → Explorer → backend → `streamlit_app.py`

Add this complete code:

```
import streamlit as st
import os
from PyPDF2 import PdfReader
from dotenv import load_dotenv

# Import project modules
from chunking import chunk_text
from embedding_service import generate_embedding
from vector_store import store_embeddings, search_embeddings
from rag_service import generate_answer

# Load environment variables
load_dotenv()

# Page configuration
st.set_page_config(
    page_title="AI Resume Assistant",
```

```
    page_icon="📄",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for better styling
st.markdown("""
    <style>
    .main-header {
        text-align: center;
        padding: 2rem 0;
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        border-radius: 10px;
        color: white;
        margin-bottom: 2rem;
    }
    .main-header h1 {
        margin: 0;
        font-size: 2.5rem;
    }
    .main-header p {
        margin: 0.5rem 0 0 0;
        font-size: 1.1rem;
        opacity: 0.9;
    }
    .section-header {
        color: #667eea;
        font-size: 1.5rem;
        margin-top: 1.5rem;
        margin-bottom: 1rem;
        border-bottom: 2px solid #667eea;
        padding-bottom: 0.5rem;
    }
    .success-box {
```

```
        background-color: #d4edda;
        border-left: 4px solid #28a745;
        padding: 1rem;
        border-radius: 5px;
        margin: 1rem 0;
    }
    .info-box {
        background-color: #e7f3ff;
        border-left: 4px solid #0066cc;
        padding: 1rem;
        border-radius: 5px;
        margin: 1rem 0;
    }
    .answer-box {
        background-color: #f8f9fa;
        border-left: 4px solid #667eea;
        padding: 1.5rem;
        border-radius: 5px;
        margin: 1rem 0;
    }
    </style>
""", unsafe_allow_html=True)

# Session state initialization
if "resume_uploaded" not in st.session_state:
    st.session_state.resume_uploaded = False
if "upload_status" not in st.session_state:
    st.session_state.upload_status = None
if "resume_filename" not in st.session_state:
    st.session_state.resume_filename = None

# Header
st.markdown("""
    <div class="main-header">
```

```
        <h1>📄 AI Resume Assistant</h1>
        <p>Upload your resume and ask questions using AI-powered search</p>
    </div>
""", unsafe_allow_html=True)

# Sidebar for file upload
st.sidebar.markdown("## 📤 Upload Resume")
st.sidebar.markdown("---")

uploaded_file = st.sidebar.file_uploader(
    "Choose a PDF file",
    type=["pdf"],
    help="Upload your resume in PDF format"
)

if uploaded_file is not None:
    if st.sidebar.button("📤 Process Resume", use_container_width=True):
        with st.sidebar.status("Processing...", expanded=True) as status:
            try:
                # Read PDF
                st.write("📖 Reading PDF...")
                reader = PdfReader(uploaded_file)
                text = ""

                for page_num, page in enumerate(reader.pages):
                    page_text = page.extract_text()
                    if page_text:
                        text += page_text

                st.write(f"✅ Extracted text from {len(reader.pages)} pages")

                # Chunk text
                st.write("✂️ Chunking text...")
                chunks = chunk_text(text)
```

```
            st.write(f"✅ Created {len(chunks)} chunks")

            # Generate embeddings
            st.write("🔢 Generating embeddings...")
            embeddings = []
            progress_bar = st.progress(0)

            for i, chunk in enumerate(chunks):
                vector = generate_embedding(chunk)
                embeddings.append(vector)
                progress_bar.progress((i + 1) / len(chunks))

            st.write(f"✅ Generated {len(embeddings)} embeddings")

            # Store embeddings
            st.write("💾 Storing in vector database...")
            store_embeddings(chunks, embeddings)
            st.write("✅ Stored successfully")

            # Update session state
            st.session_state.resume_uploaded = True
            st.session_state.upload_status = "success"
            st.session_state.resume_filename = uploaded_file.name
            st.session_state.resume_text = text
            st.session_state.chunks = chunks

            status.update(label="✅ Processing Complete", state="complete")

        except Exception as e:
            st.session_state.upload_status = "error"
            status.update(label="❌ Processing Failed", state="error")
            st.error(f"Error processing file: {str(e)}")

# Display upload status
```

```
if st.session_state.resume_uploaded:
    st.sidebar.markdown("---")
    st.sidebar.success(f"✅ **Resume Uploaded**\n\n📄 {st.session_state.resume_filename}")
    st.sidebar.info(f"📊 Chunks: {len(st.session_state.chunks)}")

# Main content area
col1, col2 = st.columns([2, 1])

with col1:
    st.markdown('<h2 class="section-header">❓ Ask Questions</h2>', unsafe_allow_html=True)

    if st.session_state.resume_uploaded:
        st.markdown("""
            <div class="info-box">
                <strong>💡 Tip:</strong> Ask questions about your resume, such as "What are my
main skills?" or "What companies have I worked for?"
            </div>
        """, unsafe_allow_html=True)

        # Question input
        question = st.text_input(
            "Enter your question:",
            placeholder="E.g., What are my technical skills?",
            help="Type a question about your resume"
        )

        # Answer button
        if question:
            if st.button("🔍 Search & Answer", use_container_width=True):
                with st.spinner("🤖 Thinking..."):
                    try:
                        # Generate embedding for question
                        question_embedding = generate_embedding(question)
```

```
            # Search for relevant chunks
            results = search_embeddings(question_embedding, top_k=3)

            # Combine context from results
            context = "\n\n".join(results['documents'][0]) if results['documents'] else ""

            if not context:
                st.warning("⚠️ No relevant information found in your resume.")
            else:
                # Generate answer
                answer = generate_answer(question, context)

                # Display answer
                st.markdown("""
                    <div class="answer-box">
                        <h3>✨ Answer</h3>
                    </div>
                """, unsafe_allow_html=True)
                st.markdown(answer)

                # Show source context
                with st.expander("📚 View Source Context", expanded=False):
                    st.markdown("**Relevant sections from your resume:**")
                    for i, chunk in enumerate(results['documents'][0], 1):
                        st.markdown(f"**Chunk {i}:**")
                        st.text(chunk)
                        st.divider()

        except Exception as e:
            st.error(f"❌ Error generating answer: {str(e)}")

else:
    st.markdown("""
        <div class="info-box">
```

```
            <strong>📌 Getting Started:</strong><br>
            1. Upload a PDF resume using the sidebar<br>
            2. Wait for processing to complete<br>
            3. Ask questions about your resume<br>
            4. Get AI-powered answers instantly
        </div>
    """, unsafe_allow_html=True)

st.markdown('<h2 class="section-header">ℹ️ About</h2>', unsafe_allow_html=True)

st.markdown("""
### How it Works

1. **Upload** - Submit your PDF resume
2. **Process** - Text is chunked and embedded
3. **Store** - Embeddings saved in vector DB
4. **Query** - Ask questions about your resume
5. **Answer** - AI generates contextual responses

### Tech Stack
- 📚 **ChromaDB** - Vector database
- 🔢 **Google Embeddings** - Text embeddings
- 🤖 **gemini-3.5-flash** - Answer generation
- ⚡ **Streamlit** - UI framework
""")

st.markdown("---")
st.markdown("""
### Tips
- Be specific with questions
- Use resume keywords
- Ask follow-up questions
- Review source context
""")
```

```
# Footer
st.markdown("---")
st.markdown(
    "<p style='text-align: center; color: #666;'>AI Resume Assistant © 2024 | Powered by Google AI</p>",
    unsafe_allow_html=True
)
```

2. Run Streamlit

Keep your FastAPI terminal running.

Open **another terminal** in VS Code.

Navigate to the backend:

cd ai-resume-assistant/backend

Activate the environment:

venv\Scripts\activate

Then run:

streamlit run streamlit_app.py

3. Open the application

Streamlit will show a local URL, usually:

http://localhost:8501

Open it in your browser.

STEP 12 — Clean Up Project Files

Now we will prepare the project properly before moving to GitHub/deployment.

1. Update `.gitignore`

Open:

VS Code → Explorer → backend → `.gitignore`

Add:

```
venv/
.env
__pycache__/
*.pyc
chroma_db/
```

This prevents your virtual environment, API key, cache files, and ChromaDB from being uploaded to GitHub.

2. Create `requirements.txt`

Your `requirements.txt` should contain:

```
fastapi
uvicorn
PyPDF2
google-genai
chromadb
streamlit
python-dotenv
python-multipart
```

3. Check your final structure

Your project should now look like:

```
ai-resume-assistant/
└── backend/
    ├── venv/
    ├── chroma_db/
    ├── main.py
    ├── streamlit_app.py
    ├── rag_service.py
    ├── chunking.py
    ├── embedding_service.py
    ├── vector_store.py
    ├── requirements.txt
    └── .env
```

`chroma_db/` will appear after you process a resume.

4. Important `.env` check

Open:

`backend/.env`

Make sure it contains:

GEMINI_API_KEY=YOUR_ACTUAL_API_KEY

Don't add quotes unless required by your environment.

STEP 14 — Create GitHub Repository

1. Create a GitHub repository

Open:

GitHub

Navigate:

GitHub → + → New repository

Enter:

Repository name:
ai-resume-assistant

Select:

Public or **Private** — your choice.

Do **not** add:

- README
- `.gitignore`
- License

Click:

Create repository

2. Open VS Code Terminal

Make sure you are in:

ai-resume-assistant/backend

Go back to the project root:

cd ..

You should now be here:

ai-resume-assistant/

3. Initialize Git

Run:

git init

4. Add project files

Run:

git add .

Because `.gitignore` contains `.env` and `venv/`, they will **not** be added.

5. Check what will be uploaded

Run:

git status

Make sure `.env` and `venv` are **not** listed for upload.

6. Create the first commit

Run:

git commit -m "Initial AI Resume Assistant"

7. Connect GitHub

Copy your repository URL from GitHub. It will look like:

https://github.com/YOUR_USERNAME/ai-resume-assistant.git

Run:

git remote add origin YOUR_GITHUB_REPOSITORY_URL

Example:

git remote add origin https://github.com/username/ai-resume-assistant.git

8. Push the project

Run:

git branch -M main
git push -u origin main

Refresh your GitHub repository.

You should now see your project files.

STEP 15 — Deploy on Streamlit Cloud

1. Open Streamlit Cloud

Go to:

Streamlit Community Cloud

Sign in using your **GitHub account**.

2. Create a new app

Click:

New app

Select:

Repository: YOUR_USERNAME/ai-resume-assistant

For the main file, enter:

backend/streamlit_app.py

3. Add your Gemini API key

Before deploying, open:

Advanced settings → Secrets

Add:

GEMINI_API_KEY = "YOUR_ACTUAL_API_KEY"

⚠️ Do **not** put the API key directly in your GitHub files.

4. Deploy

Click:

Deploy

Wait for the application to build.

If everything is correct, Streamlit will provide an online URL.

5. Open your application

Open the URL provided by Streamlit.

You should see:

📄 AI Resume Assistant

Upload your Resume

Ask a question about your resume